Informatics

Modeling Chemical Kinetics with ChemKANs: Reproduction in PyTorch and Extension to Methane

Hasan Berke Bankoglu

Master's Programme Data Science · Interdisciplinary Project

TU Wien Informatics

Supervisor: Ass. Prof. DI Dr. Markus Bösenhofer

Institute of Chemical, Environmental and Bioscience Engineering

Co-Supervisor: Univ.-Prof. Dr. Thomas Gärtner

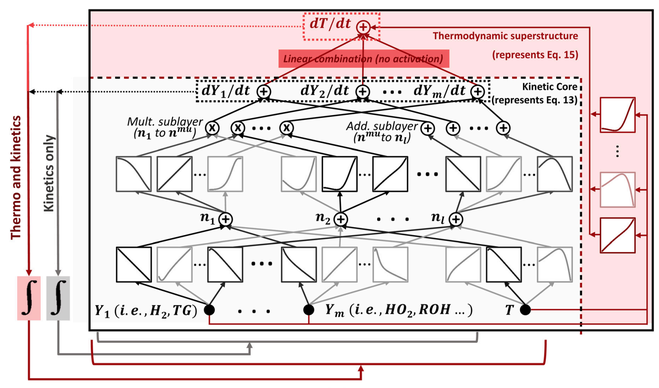
Institute of Information Systems Engineering, Research Unit Machine Learning

Context & CRISP-DM workflow

- Detailed combustion chemistry is a large, **stiff ODE system** and dominates the cost of reacting-flow simulations.
- **ChemKAN** [1] builds kinetic and thermodynamic structure into a Kolmogorov–Arnold network ODE [2, 3]; the paper reports a 344-parameter H_2–air model with 2× acceleration.
- Here: an **independent PyTorch re-implementation** of both paper benchmarks and an extension to **methane–air**.

1 Understand	2 Data	3 Model	4 Evaluate	5 Deploy
paper as the spec; RQs, KPIs in proposal	generate; train-only min–max scaling; noise	PyTorch, Tsit5, Adam; autograd or FSA	against the paper's figures and claims; MSE, ignition, speed	versioned code; config and data hash per run

ChemKAN in one picture



ChemKAN architecture, from Koenig et al. [1] (CC BY-NC 4.0). Stage 1 learns species rates with observed T(t); Stage 2 adds thermodynamics.

Data

	Biodiesel	H_2–air	CH_4–air
Chemistry	3 reactions, 6 species, isothermal	29 reactions, 9 species + T	GRI-Mech 3.0: 325 reactions, 52 modeled species (Ar excluded) + T
Conditions	20 train / 10 test; 0–15 % noise	35 train / 1 held-out; T_0 950–1200 K	35 train / 1 held-out; T_0 1400–1650 K
Window	30 s · SciPy	0.6 ms · Cantera [5]	5 ms · Cantera [5]
ChemKAN	**156** parameters	**344** parameters	**1,892** parameters

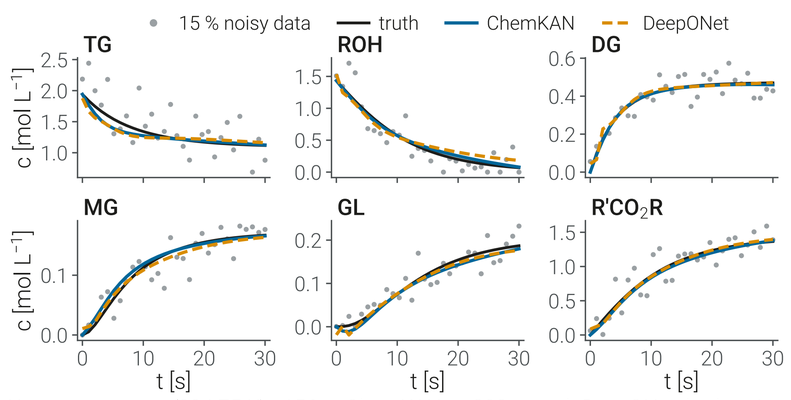
MSE: time-averaged, on min–max-scaled states (paper Eq. 18 / N_t) · gases: φ 0.5–1.5, 100 output times · one seed per run

[1] Koenig, Kim, Deng, *Phys. Chem. Chem. Phys.* 27 (2025) 17313 · [2] Koenig, Kim, Deng, KAN-ODEs, arXiv:2407.04192 · [3] Koenig, Kim, Deng, LeanKAN, *Neural Networks* 192 (2025) 107883 · [4] Lu, Jin, Karniadakis, DeepONet, arXiv:1910.03193 · [5] Goodwin et al., Cantera 3.0.0, cantera.org · [6] Owoyele, Pal, ChemNODE, *Energy and AI* 7 (2022) 100118 · [7] Ji et al., Arrhenius.jl, arXiv:2107.06172 · [8] ChemKAN authors, personal communication, 22 Sept 2026 · [9] DENG-MIT, ChemKAN released Julia biodiesel example, commit d7aa5abe

RQ1 Reproduction

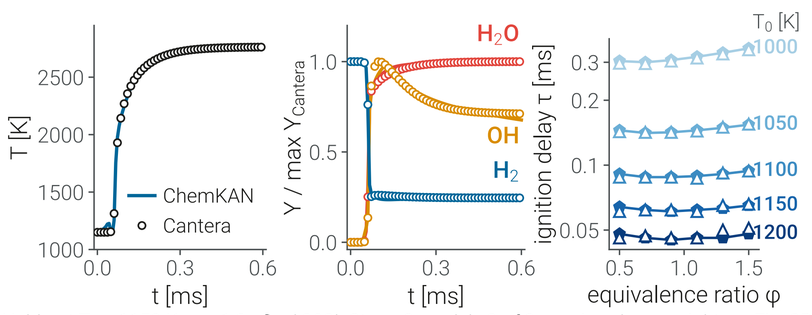
How accurately can a PyTorch reimplementation of ChemKAN [1] reproduce the predictive accuracy reported in [1] for the hydrogen combustion and biodiesel production benchmarks?

Biodiesel: model inference from 15 % noisy data



Unseen test case (334.75 K) · 156-p. ChemKAN vs 308-p. tanh DeepONet, both trained on 15 % noise · MSE on this trajectory: ChemKAN $3.4{\cdot}10^{-4}$, DeepONet $7.3{\cdot}10^{-4}$

H_2–air combustion · 344 parameters, as in the paper



Held-out T_0 = 1150 K, φ = 1.3 · final 200k Stage-2 model · 3 of 9 species shown · right: as Fig. 8B of [1], filled: Cantera, open: ChemKAN · τ = time of max dT/dt, error = $|\tau_{ChemKAN} - \tau_{Cantera}| / \tau_{Cantera}$

$3.5{\cdot}10^{-4}$
held-out MSE (1150 K, φ 1.3)

1.36 %
median ignition-delay error, 30 conditions (max 8.7 %)

$4.9{\cdot}10^{-4}$
median MSE, 441-condition grid; up to $6.1{\cdot}10^{-2}$ at 975–1000 K

What it took to reproduce

The paper text alone was not enough to reproduce its results.

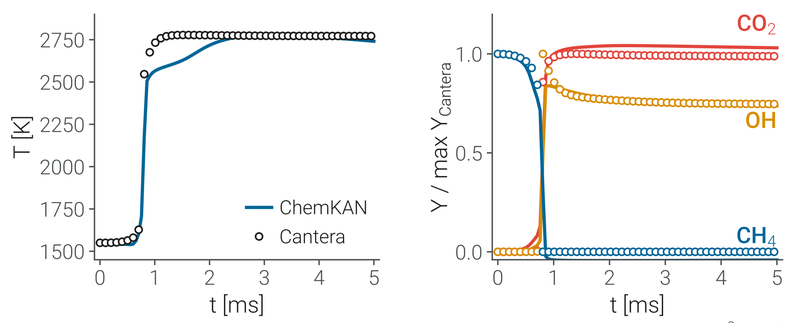
- **Paper text only:** H_2 did not ignite reliably; losses above reported levels.
- **Authors clarified or confirmed** [8]: input scaling, no base activation, 5 Gaussian RBFs per KAN edge (H_2), thermo init 10^{-5}, dense T(t), training budget.
- **Our PyTorch choice:** $\dot{\hat{u}} = F(\hat{u}, \theta) / t_{ref}$, t_{ref} = 0.6 ms (H_2), 5 ms (CH_4); no released H_2 code to confirm it.
- **Released biodiesel code** [9] vs paper: reaction 2/3, loss, solver; RHS ÷ 50.

Biodiesel claim	Paper [1]	Ours
Scaling order, train / test	1.0 / 0.6	**1.49 / 0.37**
Train-MSE increase, 0→1 % noise	$3.78{\cdot}10^{-5}$	**$4.19{\cdot}10^{-5}$**
Train-MSE increase, 0→5 % noise	$9.64{\cdot}10^{-4}$	**$1.05{\cdot}10^{-3}$**
ChemKAN clean-test MSE, 15 % / 0 %	≈ 2	**1.84**
Clean-test MSE at 15 %, DeepONet / ChemKAN	4.4	**2.73**

RQ2 Methane extension

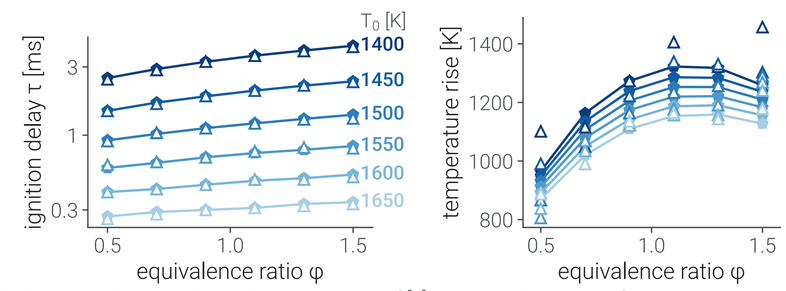
How well does ChemKAN perform in terms of predictive accuracy (test MSE) and computational efficiency (inference time relative to a traditional numerical solver) when applied to a chemical system not covered in [1], such as methane oxidation?

CH_4–air held-out trajectory · 1,892 parameters



Held-out T_0 = 1550 K, φ = 1.3 · selected epoch-180,000 Stage-2 model · MSE $1.16{\cdot}10^{-3}$ · 3 of 52 species shown · ignition timing matches; after ignition, T approaches its final value more slowly

Ignition delay and temperature rise · 36 conditions



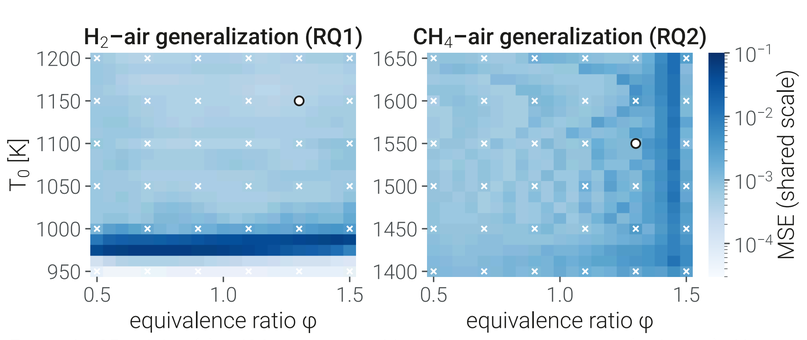
35 training + 1 held-out condition · as Fig. 8B of [1] · τ on a 0.01 ms grid · filled: Cantera, open: ChemKAN

1.2 %
median ignition-delay error (max 5.8 %), 36 conditions

$9.94{\cdot}10^{-4}$
median MSE, 36 conditions

Not converged
epoch 180k selected (loss spike at 200k); median training MSE $1.79{\cdot}10^{-3}$ (170–180k) → $1.60{\cdot}10^{-3}$ (190–200k)

Generalization over 441 initial conditions



Each grid: 35 training (×) + 406 unseen conditions, incl. the held-out case (white dot) · H_2: 200k model, CH_4: epoch 180,000 · shared colour scale shows system-normalized MSE; equal colours do not imply equal errors in physical units

$9.2{\cdot}10^{-4}$
CH_4 median MSE, 406 unseen conditions

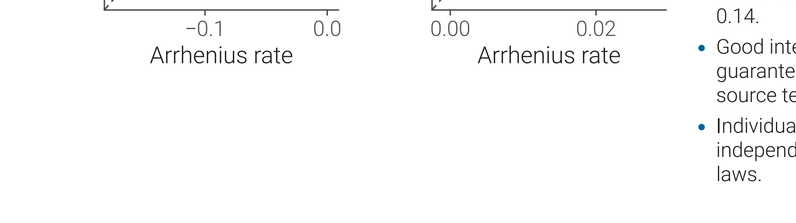
$1.27{\cdot}10^{-2}$
CH_4 maximum MSE, near φ 1.40–1.45

975–1000 K
H_2: poor generalization in the cool, ignition-sensitive band

RQ3 Comparison

How does ChemKAN compare to (a) the neural baselines used in [1] in terms of generalization to unseen initial conditions and robustness to noisy training data, and (b) traditional ODE solvers in terms of computational efficiency?

(a) Noise robustness vs. DeepONet [4]



Biodiesel · final checkpoints after 10^4 epochs · data of the report's Fig. 5A (as Fig. 5A of [1])

- Clean-test MSE: DeepONet is lower at 0–3 % and 7 % noise; ChemKAN is **2.71×** and **2.73×** lower at 10 % and 15 % (paper: 4.4× at 15 %).

(b) Inference time vs. detailed chemistry

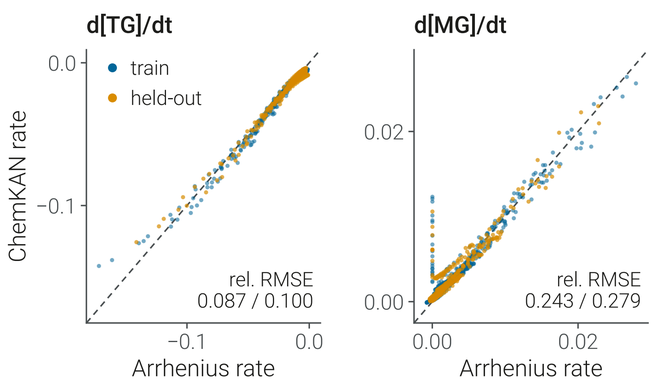
	H_2–air · 29 reactions		CH_4–air · 325 reactions	
Method	**time**	**speed-up**	**time**	**speed-up**
Cantera (CVODES) [5]	3.94 ms	reference	39.6 ms	reference
Arrhenius.jl (CVODE) [7]	10.1 ms	reference	134 ms	reference
ChemKAN · PyTorch	112 ms	0.035× / 0.090×	117 ms	0.34× / 1.1×
ChemKAN · Julia port	**1.60 ms**	**2.5× / 6.3×**	**9.93 ms**	**4.0× / 13.5×**

Median time per trajectory, Apple M1, 1 thread. PyTorch: the primary implementation used for training; Julia: inference-only port of the same trained weights, not retrained. Speed-up = reference time / ChemKAN time, vs Cantera [5] / Arrhenius.jl [7] (< 1: slower). ChemKAN at rtol/atol $10^{-6}/10^{-8}$, references at $10^{-9}/10^{-15}$: accuracy not matched.

Takeaways

- Same weights, two implementations: the Julia port runs H_2 70× faster than PyTorch (1.60 ms vs 112 ms); **the speed-up depends on the implementation**.
- Measured vs Cantera: 2.5× (H_2) and 4.0× (CH_4); two systems are too few for a trend with mechanism size.
- Biodiesel (3 reactions): the known kinetics in Julia (0.20 ms) are faster than Julia ChemKAN (0.69 ms) at tight tolerance.
- **Published, not retrained:** ChemNODE [6]: 7 networks, 637 parameters (estimate in [1]), 6 species + T, 2.3×; ChemKAN [1]: 1 network, 344 parameters, 9 species + T, 2.0×; different hardware and references.

What did ChemKAN learn? · biodiesel



Instantaneous species production rates: noise-free 156-parameter ChemKAN vs the known Arrhenius kinetics, at 600 training + 300 held-out reference states. TG and MG selected from the six-species figure in the report.

- **TG** agrees closely; **MG** shows the largest discrepancy (relative RMSE 0.243 / 0.279), with substantial error at initial states where the reference rate is zero; other species 0.09–0.14.
- Good integrated trajectories do not guarantee accurate instantaneous source terms.
- Individual KAN functions are not independently identified reaction laws.

Success criteria

- **Implementation:** PyTorch ChemKAN with kinetic core, thermodynamic superstructure, two-stage training, autograd and validated FSA; 156 / 344 / 1,892 parameters.
- **Reproduction:** biodiesel and H_2 behaviour reproduced after author clarifications (H_2 held-out MSE $3.5{\cdot}10^{-4}$, ignition error 1.36 %). Errors rise near 975–1000 K; scaling and DeepONet gaps differ, and MSE depends on the loss convention.
- **Extension:** methane–air: held-out MSE $1.16{\cdot}10^{-3}$; Julia port 4.0× faster than Cantera; training not converged.
- **Comparative analysis:** DeepONet reconstructed and retrained (0–15 % noise); published ChemNODE only (not retrained); Cantera and Arrhenius.jl timed.
- **Interpretability:** KAN functions visualized; learned biodiesel rates approximate the known kinetics at sampled states; MG has the largest discrepancy; functions not identified with reactions.

Conclusions

- From the paper text alone, key results (H_2 ignition, loss levels) **could not be reproduced**: several details were unstated or inconsistent.
- With the authors' clarifications and released code, the PyTorch ChemKAN **reproduces the main biodiesel and H_2 behaviour**; quantitative differences remain.
- ChemKAN **transfers to methane–air** (52 species, 325 reactions): $9.2{\cdot}10^{-4}$ median MSE on 406 unseen conditions, but training had not converged.
- The **speed-up depends on the implementation** and on the tolerances compared: Julia 2.5× (H_2) / 4.0× (CH_4) faster than Cantera; PyTorch slower.
- Biodiesel rate parity: **good trajectory fits, imperfect instantaneous rates** (MG).

Project website: bnkglu.github.io/chemkan-tuwien
Code, data and runs: github.com/bnkglu/chemkan-tuwien